Table of Contents

..... 1
Part A - Modeling 1
Part B - Kalman Filter Implementation 1
Part B(4) - Innovation Sequence Plot 4
Part C - Ground Truth Comparison 4
Part E - Parameter Sensitivity (Trial 1) 7
Part E- RMSE for different Q values 8


```
clear; close all; clc;
```

Part A - Modeling

```
sigma = 0.5;
R = eye(2) * (sigma^2);
Q = eye(4);
Ts = 1;

A_d = [1 0 1 0;
       0 1 0 1;
       0 0 1 0;
       0 0 0 1];
C_d = [1 0 0 0;
       0 1 0 0];

x0 = [0; 0; 0; 0];
% High uncertainty in velocity, moderate in position
P0 = diag([1, 1, 10, 10]);
```

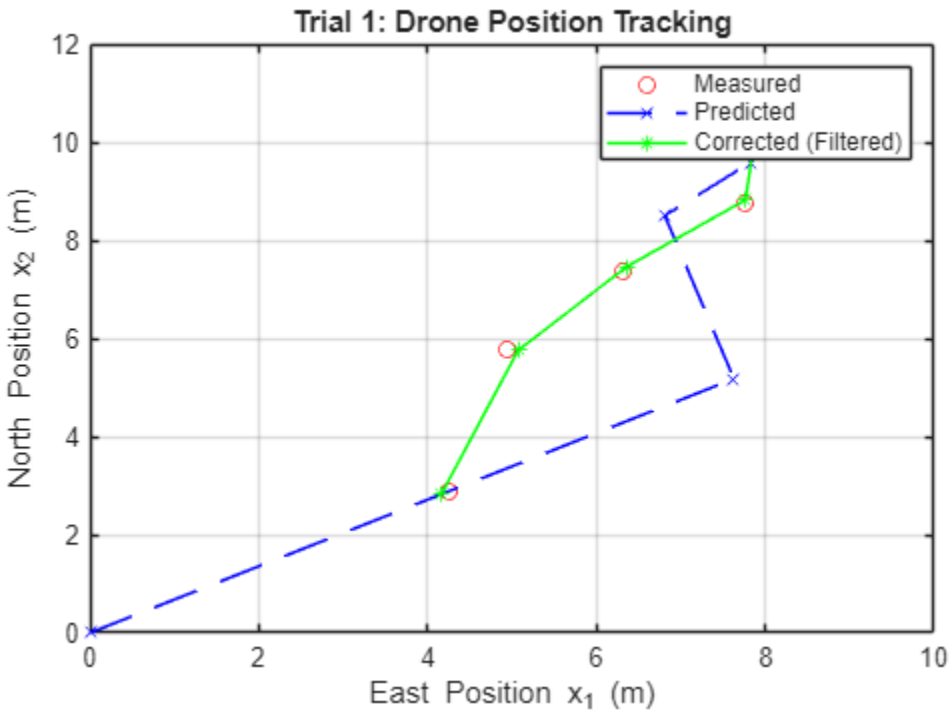
Part B - Kalman Filter Implementation

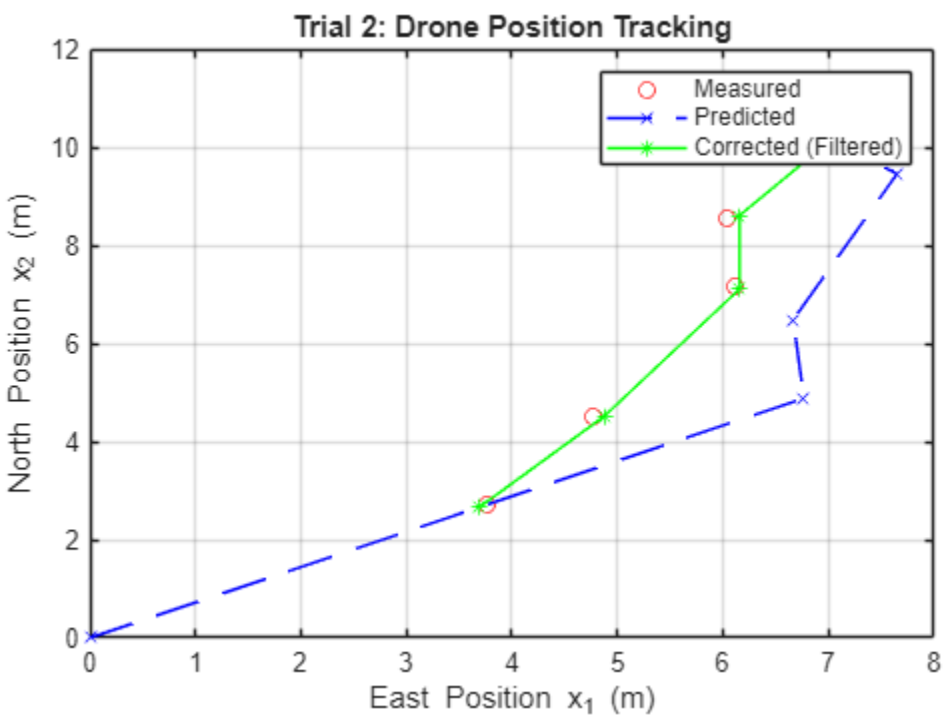
```
Y1 = [4.2484 4.9309 6.3238 7.7615 7.8829;
      2.8829 5.7896 7.3837 8.7653 11.2713];
Y2 = [3.7683 4.7671 6.1210 6.0434 7.1375;
      2.7189 4.4936 7.1571 8.5460 10.2938];

trials = {Y1, Y2};
N = size(Y1, 2);
tspace = 1:Ts:N;

all_x_pred = cell(1,2);
all_x_corr = cell(1,2);
all_innovations = cell(1,2);

for t_idx = 1:2
    Y = trials{t_idx};

    x_pred = zeros(4, N);
```

```
    x_corr = zeros(4, N);
    innovations = zeros(2, N);

    x_curr = x0;
    P_curr = P0;

    % Kalman Filter Loop
    for k = 1:N
        % 1. PREDICT STEP
        x_p = A_d * x_curr;
        P_p = A_d * P_curr * A_d' + Q;

        % Store predictions
        x_pred(:, k) = x_p;

        % 2. UPDATE STEP
        y_k = Y(:, k);
        inn = y_k - C_d * x_p;                  % Innovation
        S = C_d * P_p * C_d' + R;               % Innovation covariance
        K = P_p * C_d' / S;                     % Kalman Gain

        x_curr = x_p + K * inn;                 % Corrected state
        P_curr = (eye(4) - K * C_d) * P_p;      % Corrected covariance

        % Store corrections and innovations
        x_corr(:, k) = x_curr;
        innovations(:, k) = inn;
    end

    all_x_pred{t_idx} = x_pred;
    all_x_corr{t_idx} = x_corr;
    all_innovations{t_idx} = innovations;

    figure('Position', [300,200,600,400]);
    plot(Y(1,:), Y(2,:), 'ro', 'DisplayName', 'Measured'); hold on;
    plot(x_pred(1,:), x_pred(2,:), 'bx--', 'DisplayName', 'Predicted');
    plot(x_corr(1,:), x_corr(2,:), 'g*-', 'DisplayName', 'Corrected
(Filtered)');
    title(['Trial ', num2str(t_idx), ': Drone Position Tracking']);
    xlabel('East Position x_1 (m)'); ylabel('North Position x_2 (m)');
    legend; grid on;

    fprintf('Trial %d Final Estimated State x_5:\n', t_idx);
    disp(x_corr(:, end));
end
```

```
Trial 1 Final Estimated State x_5:
    7.9773
   11.2166
    0.7277
    2.0936

Trial 2 Final Estimated State x_5:
    7.1122
```

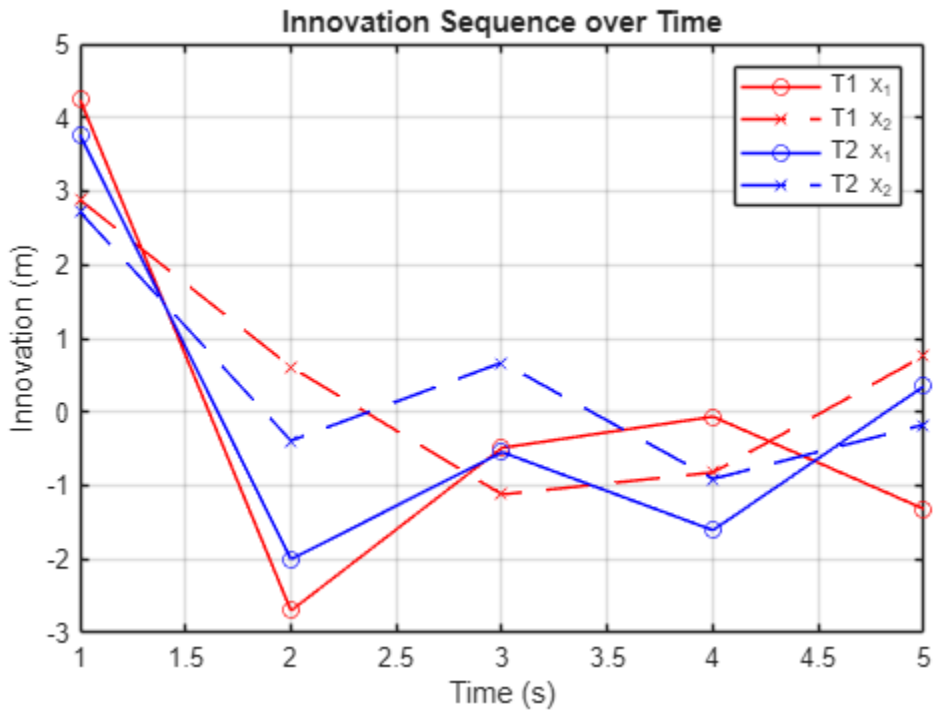
```
10.3060
 0.8161
 1.7623
```





Part B(4) - Innovation Sequence Plot

```
figure('Position', [300,200,600,400]);
plot(tspace, all_innovations{1}(1,:), 'r-o', tspace, all_innovations{1}
(2,:), 'r--x'); hold on;
plot(tspace, all_innovations{2}(1,:), 'b-o', tspace, all_innovations{2}
(2,:), 'b--x');
title('Innovation Sequence over Time');
xlabel('Time (s)'); ylabel('Innovation (m)');
legend('T1 x_1', 'T1 x_2', 'T2 x_1', 'T2 x_2'); grid on;
```

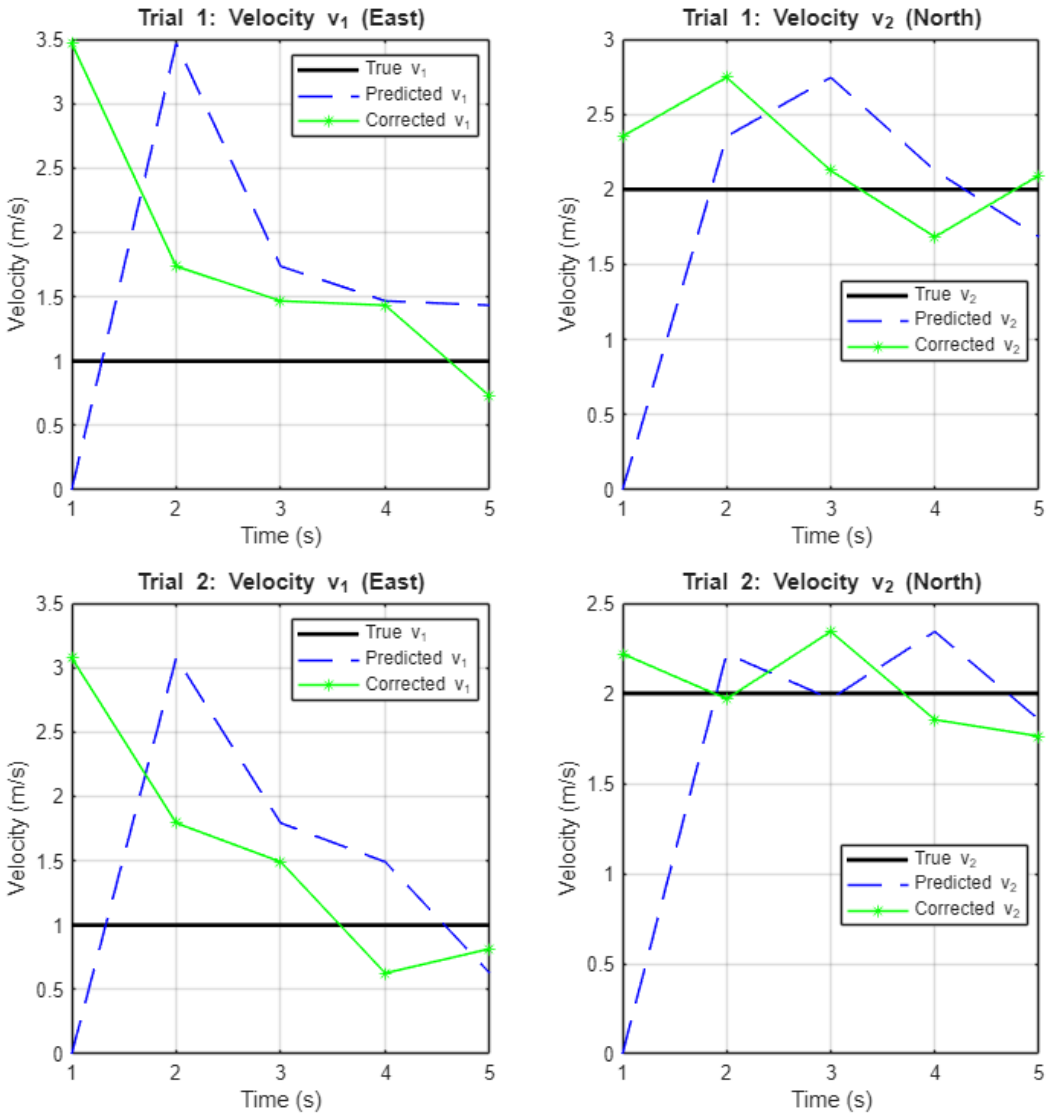


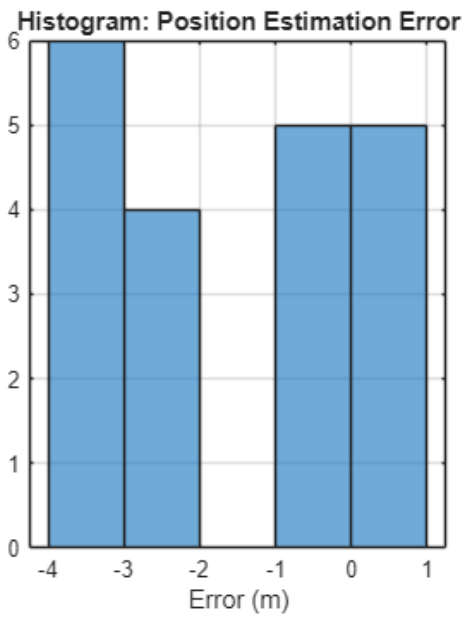
Part C - Ground Truth Comparison

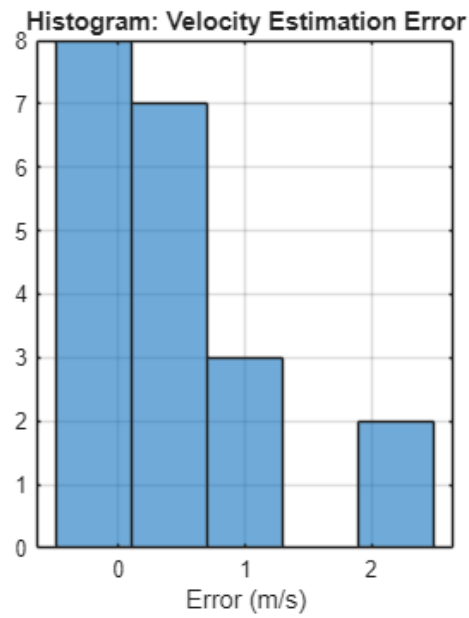
```
x1_true = tspace + 3;
x2_true = 2*tspace + 4;
v1_true = 1 * ones(1, N);
v2_true = 2 * ones(1, N);

for t_idx = 1:2
    figure('Position', [400,300,900,400]);

    subplot(1,2,1);
    plot(tspace, v1_true, 'k-', 'LineWidth', 2, 'DisplayName', 'True v_1');
hold on;
    plot(tspace, all_x_pred{t_idx}(3,:), 'b--', 'DisplayName', 'Predicted
v_1');
    plot(tspace, all_x_corr{t_idx}(3,:), 'g*-', 'DisplayName', 'Corrected
v_1');
    title(['Trial ', num2str(t_idx), ': Velocity v_1 (East)']);
```

```
    xlabel('Time (s)'); ylabel('Velocity (m/s)');
    legend('Location', 'best'); grid on;

    subplot(1,2,2);
    plot(tspace, v2_true, 'k-', 'LineWidth', 2, 'DisplayName', 'True v_2');
hold on;
    plot(tspace, all_x_pred{t_idx}(4,:), 'b--', 'DisplayName', 'Predicted
v_2');
    plot(tspace, all_x_corr{t_idx}(4,:), 'g*-', 'DisplayName', 'Corrected
v_2');
    title(['Trial ', num2str(t_idx), ': Velocity v_2 (North)']);
    xlabel('Time (s)'); ylabel('Velocity (m/s)');
    legend('Location', 'best'); grid on;
end

pos_errors = [];
vel_errors = [];

for t_idx = 1:2
    x_c = all_x_corr{t_idx};

    pos_err = [x_c(1,:) - x1_true; x_c(2,:) - x2_true];
    vel_err = [x_c(3,:) - v1_true; x_c(4,:) - v2_true];

    pos_errors = [pos_errors, pos_err];
    vel_errors = [vel_errors, vel_err];
end

figure('Position', [500,300,800,400]);

subplot(1,2,1);
histogram(pos_errors(:), 5);
title('Histogram: Position Estimation Error');
xlabel('Error (m)');
grid on;

subplot(1,2,2);
histogram(vel_errors(:), 5);
title('Histogram: Velocity Estimation Error');
xlabel('Error (m/s)');
grid on;

pos_rmse = sqrt(mean(pos_errors(:).^2));
vel_rmse = sqrt(mean(vel_errors(:).^2));
fprintf('\n--- Part C: RMSE ---\n');
fprintf('Position RMSE: %.4f m\n', pos_rmse);
fprintf('Velocity RMSE: %.4f m/s\n', vel_rmse);
```

```
--- Part C: RMSE ---
Position RMSE: 2.2198 m
Velocity RMSE: 0.8221 m/s
```

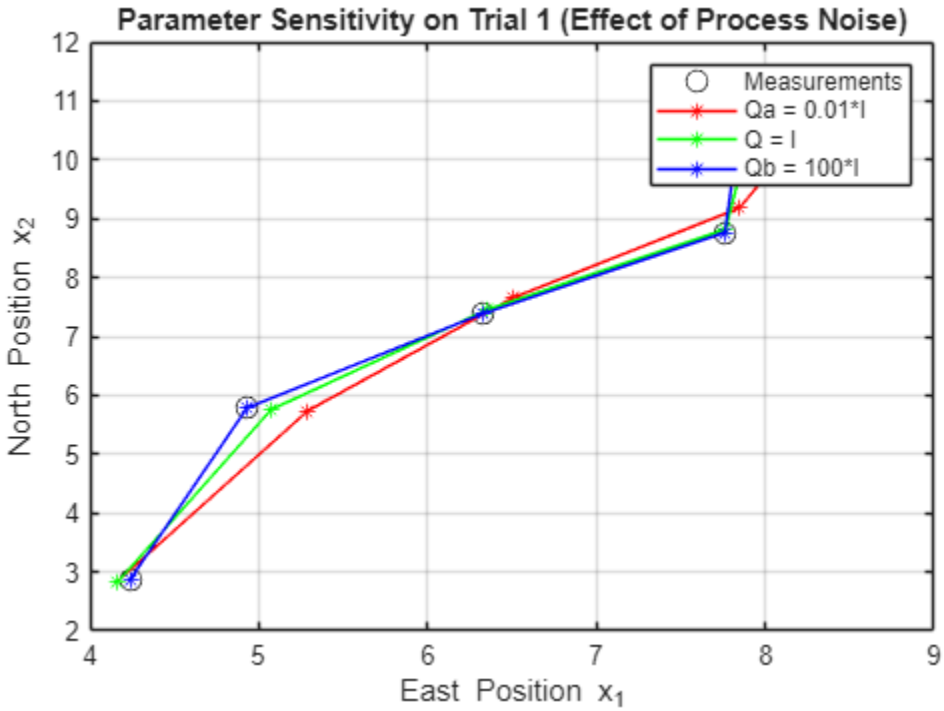

Trial 1: Velocity v1 (East)
Trial 1: Velocity v2 (North)
Trial 2: Velocity v1 (East)
Trial 2: Velocity v2 (North)
True v1
Predicted v1
Corrected v1
True v2
Predicted v2
Corrected v2
Velocity (m/s)
Time (s)





Part E - Parameter Sensitivity (Trial 1)

```
Q_vals = {0.01 * eye(4), eye(4), 100 * eye(4)};
Q_labels = {'Qa = 0.01*I', 'Q = I', 'Qb = 100*I'};
colors = {'r', 'g', 'b'};

figure('Position', [300,200,600,400]);
plot(Y1(1,:), Y1(2,:), 'ko', 'MarkerSize', 8, 'DisplayName',
'Measurements'); hold on;

for i = 1:length(Q_vals)
    Q_test = Q_vals{i};
    x_curr = x0; P_curr = P0;
    x_corr_test = zeros(4, N);

    for k = 1:N
        x_p = A_d * x_curr;
        P_p = A_d * P_curr * A_d' + Q_test;
        y_k = Y1(:, k);
        K = P_p * C_d' / (C_d * P_p * C_d' + R);
        x_curr = x_p + K * (y_k - C_d * x_p);
        P_curr = (eye(4) - K * C_d) * P_p;
        x_corr_test(:, k) = x_curr;
    end
    plot(x_corr_test(1,:), x_corr_test(2,:), [colors{i} '*-'],
'DisplayName', Q_labels{i});
end
title('Parameter Sensitivity on Trial 1 (Effect of Process Noise)');
xlabel('East Position x_1'); ylabel('North Position x_2');
legend; grid on;
```



Part E- RMSE for different Q values

```
Q_vals = {0.01 * eye(4), eye(4), 100 * eye(4)};
Q_names = {'Qa (0.01)', 'Original (1.0)', 'Qb (100.0)'};
trials = {Y1, Y2};

rmse_results = zeros(length(Q_vals), 2);


t_vec = 1:5;
xtrue = [t_vec + 3; 2*t_vec + 4];

for t_idx = 1:2
    Y_current = trials{t_idx};

    for q_idx = 1:length(Q_vals)
        Q_test = Q_vals{q_idx};

        x_curr = x0;
        P_curr = P0;
        x_corr_history = zeros(2, 5);

        for k = 1:5
            % Predict
            x_p = A_d * x_curr;
            P_p = A_d * P_curr * A_d' + Q_test;

            % Update
            y_k = Y_current(:, k);
```

```
            K = P_p * C_d' / (C_d * P_p * C_d' + R);
            x_curr = x_p + K * (y_k - C_d * x_p);
            P_curr = (eye(4) - K * C_d) * P_p;

            x_corr_history(:, k) = x_curr(1:2); % Grab positions
        end

        errors = x_corr_history - xtrue;
        rmse_results(q_idx, t_idx) = sqrt(mean(errors(:).^2));
    end
end

fprintf('\n--- Position RMSE Comparison ---\n');
fprintf('%-15s | %-10s | %-10s\n', 'Q Value', 'Trial 1', 'Trial 2');
fprintf('----------------------------------------------\n');
for i = 1:3
    fprintf('%-15s | %-10.4f | %-10.4f\n', Q_names{i}, rmse_results(i,1),
rmse_results(i,2));
end
```

```
--- Position RMSE Comparison ---
Q Value         | Trial 1    | Trial 2
----------------------------------------------
Qa (0.01)       | 1.9371     | 2.3439
Original (1.0)  | 2.0038     | 2.4165
Qb (100.0)      | 2.0043     | 2.4218
```

Published with MATLAB® R2025b